

Aula 08 – Estruturas de dados em Python 3

Parte 1 – Revisão Teórica

Tipos de variáveis em Python

Como na maioria das linguagens temos o conceito de variáveis e tipos de dados e apesar da linguagem ser tipada dinamicamente, ela é **fortemente tipada** como veremos em breve.

Tipo	Seção	Exemplos
bool		True ou False
int	Números	3
float	Números	3.3
str	Strings	'João da Silva' ou "João da Silva"
list	Listas	[1, 2, 'ab']
dict	Dicionários	{'nome': 'João da Silva', 'idade': 21}
NoneType		None

Tabela 1. Tipos básicos de dados em Python

Estrutura de dados é qualquer meio utilizado para armazenar e recuperar dados

- a) **Homogênea**
- b) **Heterogênea**

Principais estruturas de dados

- 1) **Lista**
- 2) **Tupla**
- 3) **dicionário**
- 4) **Array/Matriz/vetor**
- 5) **Set(grupo)**
- 6) **Pilha**
- 7) **Arvore**

Estruturas de dados são agrupamentos que permitem representar e organizar informações muito além do que seria possível com tipos de dados simples.

Python possui um poderoso conjunto de estruturas de dados (também conhecidas como **containers**) embutidas na linguagem.

Algumas delas foram vistas anteriormente: **listas e tuplas são casos especiais** de estruturas de dados.

Estrutura de dados tipo Pilha (Stack – Técnica LIFO)

Estrutura de dados onde a ordenação natural é estabelecida pela ordem de entrada. O primeiro item da pilha sempre será o último elemento adicionado.

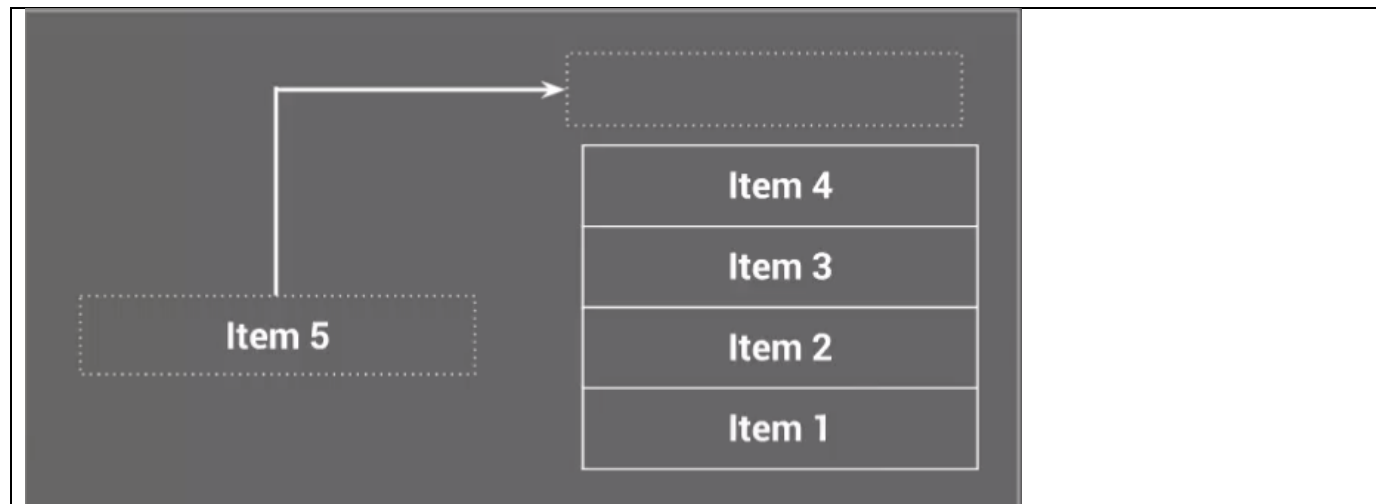


Figura 3

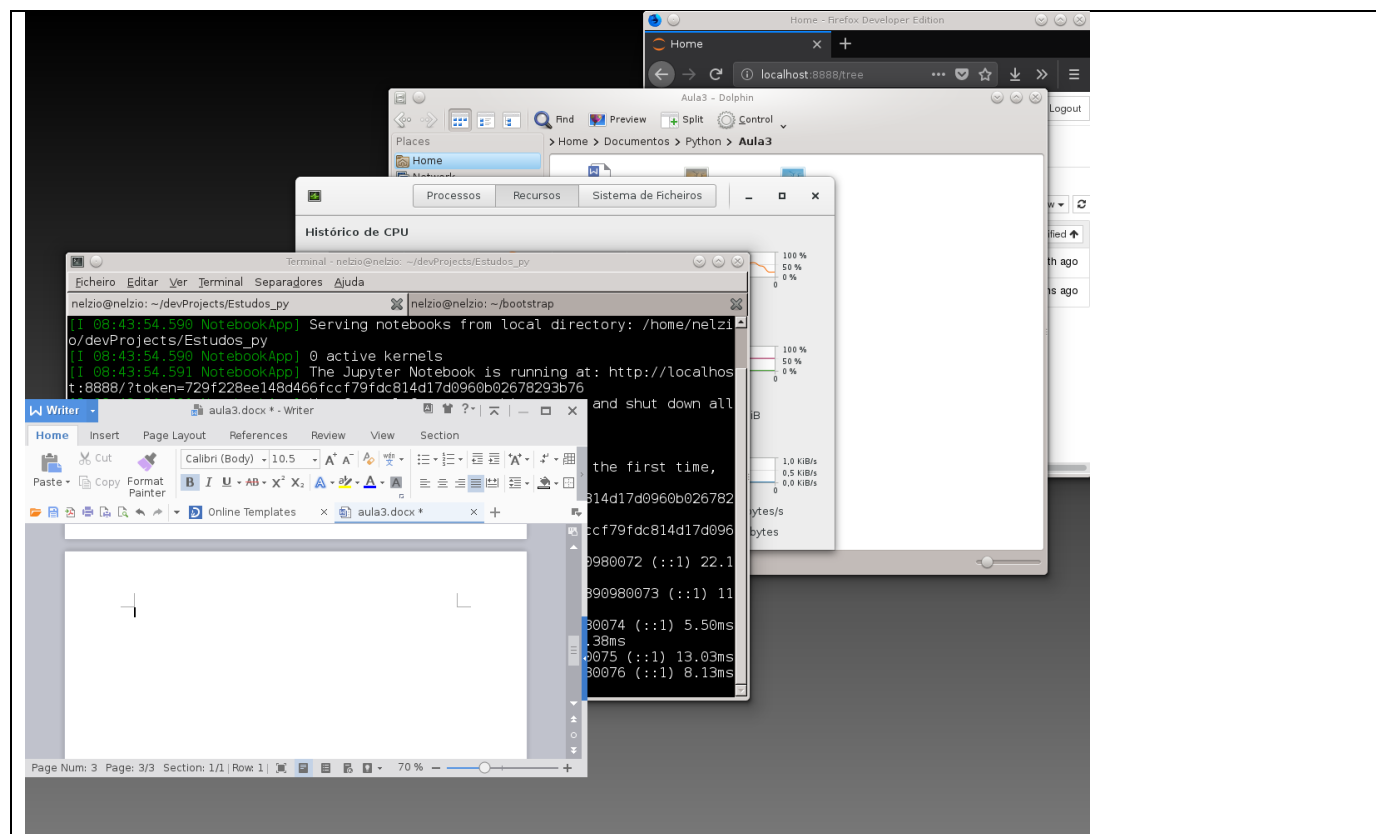


Figura 4: Exemplo de janelas de programas abertos no SO

Na pilha, o último item a ser adicionado será o primeiro a ser lido. Na imagem 4 está um exemplo, janelas do computador. A última janela a se abrir é a primeira a ser fechada.

Estrutura de dados Array

Array é uma estrutura de dados estática para manipulação de um número finito de elementos de um mesmo tipo.



Array pode ser chamado de **matriz** ou **vetor**

No python não é comum usar o array, para a maior parte dos trabalhos iremos usar a estrutura do **tipo list** e **tuple**, que será o nosso tema do próximo artigo. Mas é obrigatório que todo programador conheça a estrutura array.

Todo array possui um índice numérico inteiro. O primeiro elemento de um array possui o índice igual a zero por convenção.

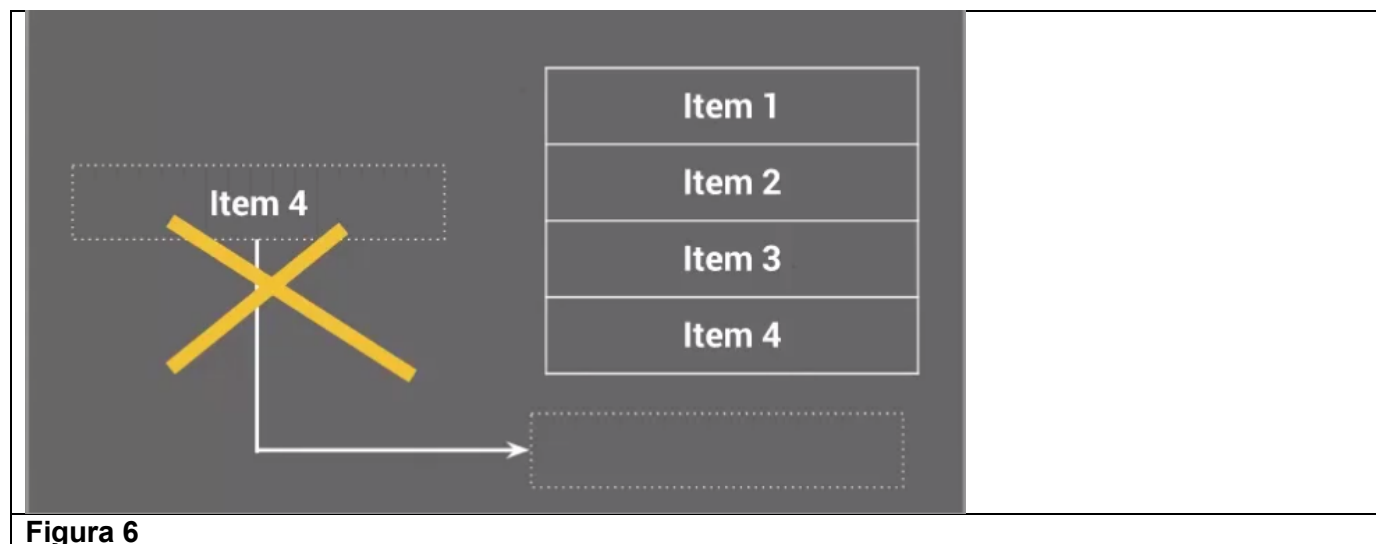
O último elemento do array tem como índice a quantidade de elementos menos 1.

Estrutura de dados Set (Conjunto)

Estrutura de dados semelhante a uma lista. **Um set tem como princípio conter uma lista de valores diferentes.** Não podemos replicar valores na estrutura de dados do tipo Set.

Conjuntos em Python possuem as mesmas características de seus equivalentes matemáticos:

- a) não possuem ordem definida nem elementos duplicados.**
- b) operações como união, interseção e diferença.**



Declaração

```
conjunto = {1, 2, 3}
# declaração de conjunto vazio
conjunto = set()
```

set() também pode ser usado para gerar um conjunto a partir de uma sequência.

Exemplos:

```
# teste de pertencimento
cesta = {'maça', 'banana', 'maça', 'laranja'}
'laranja' in cesta

# remoção de itens duplicados de uma lista
nova_lista = list(set(uma_lista))
# remoção de caracteres repetidos em uma string
set('abracadabra')
```

Métodos

Método	Resultado
.add(n)	Adiciona um elemento ao conjunto
.union(conj)	Retorna um conjunto com os elementos de ambos
.intersection(conj)	Retorna conjunto com elementos comuns a ambos
.difference(conj)	Retorna conjunto com a diferença entre ambos
.symmetric_difference(conj)	Retorna conjunto com elementos não comuns
.isdisjoint(conj)	Retorna True se a interseção é vazia
.issubset(conj)	Retorna True se um conjunto está contido no outro
.issuperset(conj)	Retorna True se um conjunto contém o outro
.copy()	Retorna uma cópia do conjunto
len()	Retorna o número de elementos do conjunto (cardinalidade)

Alguns desses métodos possuem um operador equivalente:

Operador	Equivalente
A B	A.union(B)
A & B	A.intersection(B)
A - B	A.difference(B)
A ^ B	A.symmetric_difference(B)
A < B	A.issubset(B)
A > B	A.issuperset(B)

Estrutura de dados do tipo Dicionário

Todos os tipos de dados compostos que estudamos em detalhes até agora — **strings**, **listas** e **tuplas** — são **coleções sequenciais**.

- ✓ **Dicionário** é um tipo diferente de coleção.
- ✓ Ele é um **tipo de mapeamento (associação)** nativo do Python. Um mapa é uma coleção **associativa desordenada**.
- ✓ A associação (mapeamento) é feita a partir de uma **chave**, que pode ser qualquer tipo imutável, para um **valor**, que pode ser qualquer objeto de dados do Python.



PYTHON: TRABALHANDO COM DICIONÁRIOS

A CRIANDO UMA TUPLA É BEM SIMPLES
BASTA COLOCAR ENTRE PARENTÊSES ()

```
FILMES = ( 'SENHOR DOS ANEIS' , '1974' )  
PRINT(FILMES)
```

B PARA NÃO PRECISARMOS DE UMA VARIÁVEL PARA CADA FILME,
PODEMOS COLOCÁ-LOS DIRETO EM UMA LISTA DE FILMES

```
LISTA_FILMES = [ ( 'SENHOR DOS ANEIS' , '1974' ) , ( 'MATRIX' , '1999' ) ]  
PRINT (LISTA_FILMES)
```

A ('SENHOR DOS ANEIS', '1974')

B [('SENHOR DOS ANEIS', '1974'), ('MATRIX', '1999')]

SAÍDA

Isto significa que os itens na coleção estão **ordenados da esquerda para a direita** e eles usam **números inteiros** como índices para acessar os valores que eles contêm.

Os dicionários Python são **estruturas de dados** nas quais é realizada uma associação entre uma **chave (key)** e um valor correspondente (**value**).

Estrutura de dados onde cada elemento está associado a uma chave que pode ser de qualquer tipo.

- ✓ A **chave (key)** é geralmente uma **string** ou **inteiro**, e
- ✓ os **valores** podem ser qualquer **tipo de objeto** Python.
- ✓ Assim como conjuntos (set), dicionários também não possuem uma ordem definida.



Figura 7

Os conjuntos **chave-valor** que formam um dicionário são separados por `:`, e cada par é separado do outro por `,`. Os elementos de um dicionário são delimitados por `{}`.

Declaração

```
dicionario = {
    '1ra_chave': '1ro_valor',
    '2a_chave': '2do_valor',
    '3ra_chave': '3fo_valor'
}
```

Métodos

Método	Resultado
<code>.get(chave)</code>	Retorna o valor associado ou None caso não exista
<code>.pop(chave)</code>	Retorna o valor associado e remove o par do dicionário
<code>.clear()</code>	Apaga todos os pares chave-valor
<code>.copy()</code>	Retorna uma cópia do dicionário
<code>.items()</code>	Retorna uma <i>visualização</i> dos pares do dicionário
<code>.keys()</code>	Retorna uma <i>visualização</i> das chaves do dicionário
<code>.values()</code>	Retorna uma <i>visualização</i> dos valores do dicionário

```
# declaração de um dicionario vazio
dicionario = {}
gato = {'tamanho': 'grande',
        'cor': 'cinza'
}
```

Acesso

```
gato['cor']
```

Dicionários usam uma técnica conhecida como *hashing* na associação dos pares, o que otimiza a busca de itens em relação a outras sequências.

Adição

```
dicionario['nova_chave'] = 'novo_valor'
```

Remoção

```
del dicionario['nova_chave']
```

Iteração

Para iterar por um dicionário, o par **chave-valor** pode ser obtido simultaneamente com o uso do método `items()`.

```
knight = {'gallahad': 'the pure',  
          'robin': 'the brave'}  
  
for chave, valor in knight.items():  
    print(chave, valor)
```

Como exemplo, vamos criar um dicionário para traduzir palavras em inglês para Espanhol. Para este dicionário, as chaves são strings.

Uma maneira de criar um dicionário é começar com o dicionário vazio e adicionar **pares chave-valor**. O dicionário vazio é denotado `{ }`

```
english2spanish = {}  
english2spanish['one'] = 'uno'  
english2spanish['two'] = 'dos'  
english2spanish['three'] = 'tres'  
english2spanish['four'] = 'cuatro'  
  
print("english2spanish")
```

Listagem 1

A primeira atribuição cria um dicionário chamado `english2spanish`. As outras atribuições adicionam novos pares chave-valor para o dicionário. O lado esquerdo define o dicionário e a chave a ser associada. O lado direito define o valor a ser associado a essa chave.

Nós podemos imprimir o valor atual do dicionário da maneira usual. Os pares chave-valor do dicionário são separados por vírgulas. cada par contém uma chave e um valor separado por dois pontos.

A ordem dos pares pode não ser a que você esperava. O Python usa algoritmos complexos, projetados para acesso muito rápido, para determinar onde os pares chave-valor são armazenadas em um dicionário. Para os nossos propósitos, podemos pensar que esta ordenação é imprevisível.

Outra maneira de criar um dicionário é fornecer uma lista de pares chave-valor usando a mesma sintaxe que a saída anterior.

```
english2spanish = {'three': 'tres',
                   'one': 'uno',
                   'two': 'dos',
                   'four': 'cuatro'
                   }

print(english2spanish)
```

Listagem 2

Não importa em que ordem escrevemos os **pares (chave e valor)**.

Os valores em um dicionário são **acessados com chaves**, não com índices, por isso **não** há necessidade de se **preocupar com a ordenação**.

O exemplo mostra como usar uma chave para pesquisar o valor correspondente.

```
english2spanish = {
    'three': 'tres',
    'one': 'uno',
    'two': 'dos',
    'four': 'cuatro'
}

print(english2spanish)

valor = english2spanish['two']
print(valor)
```

Listagem 3

A chave 'two' resulta no valor 'dos'.

FUNÇÕES DA Estrutura do tipo Dicionário

Operações com dicionários

O comando **del remove** um **par chave-valor** de um dicionário.

Por exemplo, o seguinte dicionário (estoque) contém os nomes de várias frutas e o número de cada fruta em estoque.

Se alguém compra todas as peras, podemos remover a entrada do dicionário.

```
#dicionario = {'1ra_chave': '1ro_valor', '2a_chave': '2do_valor'}
#Chave = Frutas
#Valor = Quantidade

estoqueFrutas1 = {'kiwis': 430,
                  'bananas': 312,
                  'laranjas': 525,
                  'peras': 217
                  }
```

```
estoqueFrutas2 = {'mangas': 130,
                  'abacaxis': 150,
                  'uvas': 325,
                  'ameixas': 517
                  }

print("Conteúdo completo do dicionário (estoque) = " , estoqueFrutas1)
print("Conteúdo completo do dicionário (estoque) = " , estoqueFrutas2)
```

Listagem 4: Estoque de frutas (nome e quantidade) utilizando a estrutura de dados tipo Dicionário

Global variável estoque	"peras"	217
	"kiwis"	430
	"laranjas"	525
	"bananas"	312

del(x): Um elemento pode ser removido de uma lista utilizando a função **del()** e enviando como parâmetro, qual elemento da lista nós desejamos que seja removido.

```
estoque = {'kiwis': 430,
           'bananas': 312,
           'laranjas': 525,
           'peras': 217
           }

print("Conteúdo completo do dicionário (estoque) = " , estoque)

del estoque['peras']

print("Conteúdo do dicionário (estoque), após a exclusão do chave/valor = pera " , estoque)
```

Listagem 5: Se alguém compra todas as peras, podemos remover a entrada do dicionário.

Dicionários também são mutáveis. Como vimos antes, com listas, isso significa que o dicionário pode ser modificado pela referência a uma associação no lado esquerdo de um comando de atribuição. No exemplo anterior, em vez de excluir a entrada para peras, poderíamos ter definido o inventário para 0.

```
#dicionario = {'1ra_chave':'1ro_valor', '2a_chave':'2do_valor'}
#Chave = Frutas
#Valor = Quantidade

estoqueFrutas1 = {'kiwis': 430,
                  'bananas': 312,
                  'laranjas': 525,
                  'peras': 217
                  }

print("Conteúdo completo do dicionário (estoque) = " , estoqueFrutas1)

del estoqueFrutas1['peras']

print("Conteúdo do dicionário (estoque), após a exclusão do chave/valor = pera " , estoqueFrutas1)
estoqueFrutas1 ['peras'] = 0
```

```
print("Estoque de frutas atual = ", estoque)
```

Listagem 6: Em vez de excluir a entrada para peras, poderíamos ter definido o inventário para 0.

len(x): semelhante a forma como utilizamos em outras estruturas, para nos conhecermos a quantidade de elementos contidos numa lista, basta nós utilizarmos a função **len()** e passarmos como parâmetro a lista que desejarmos conhecer o tamanho.

Similarmente, um novo carregamento de bananas pode ser tratado assim.

```
#dicionario = {'1ra_chave':'1ro_valor', '2a_chave':'2do_valor'}
#Chave = Frutas
#Valor = Quantidade

estoqueFrutas1 = {'kiwis': 430,
                  'bananas': 312,
                  'laranjas': 525,
                  'peras': 217
                  }

print("Conteúdo completo do dicionário (estoque) = " , estoqueFrutas1)

estoqueFrutas1['bananas'] = estoqueFrutas1['bananas'] + 200

numItens = len(estoqueFrutas1)
print("Numero de itens = " , numItens)
print("Estoque de bananas = " , estoqueFrutas1['bananas'])

print("Estoque total de frutas = " , estoqueFrutas1)
```

Listagem 7

Global
variables
estoque

"peras"	217
"kiwis"	430
"laranjas"	525
"bananas"	512

Observe que há agora 512 bananas — o dicionário foi modificado. Note também que a **função len()** também funciona em dicionários. Ela retorna o número de pares chave-valor:

Um dicionário possui mesmas funções, características de uma lista, porém, em um dicionário cada valor possui uma chave.

Um dicionário **não aceita que dois elementos possuam a mesma chave**. O dicionário é parecido com a estrutura de dados do tipo set. A diferença é que no dicionário temos o esquema **chave-valor**.

A grande diferença do **dicionário** para as listas, é que com os dicionários, é **nossa responsabilidade a construção e o gerenciamento do índice** e também, nós podemos utilizar como índice qualquer objeto não mutável.

Ou seja, nós podemos definir que cada item terá uma cadeia de caracteres vinculado, logo, para acessar este item, deve-se informar qual a sequência de caracteres desejado. Também podemos associar um

número como índice ou então, nós podemos definir que alguns elementos terão vinculados um número e outros uma string ou mesmo, uma tupla.

Nós temos uma estrutura chave-valor, e cada chave sempre estará relacionada a um valor qualquer. Logo, podemos dizer que um dicionário pode funcionar como uma lista, mas uma lista não consegue trabalhar como um dicionário.

A estrutura de dicionário é uma estrutura abstrata, assim, podemos estende-la e implementarmos uma lista simples, uma tupla ou então, a estrutura de chave-valor que desejarmos.

CHAVES INVÁLIDAS

Em um dicionário, nós podemos definir como chave de um elemento, qualquer estrutura imutável, como exemplo, uma tupla ou então, uma string. Porém, se nós tentarmos atribuir como chave uma lista, não será possível e isso acontece, porque internamente, um dicionário utiliza um valor que é obtido através de uma chave. Logo, se nós tivermos uma estrutura mutável, o valor que o dicionário utilizou como índice terá sido mudado e então, o armazenamento chave-valor não mais funcionará.

OPERADORES in e not in

Quando precisarmos saber se uma letra está contida em um conjunto, podemos utilizar o operador "in", que numa tradução, significa: "em", "dentro de", "está em" ou "está contido em".

Da mesma forma, nós podemos verificar se um elemento não está contido, utilizando a negação do operador in, que é: "not in": "não está dentro", "não pertence ao conjunto de".

```
#dicionario = {'1ra_chave':'1ro_valor', '2a_chave':'2do_valor'}
#Chave = Frutas
#Valor = Quantidade

estoqueFrutas1 = {'kiwis': 430,
                  'bananas': 312,
                  'laranjas': 525,
                  'peras': 217
                  }

print ("Tem peras no estoque de Frutas 1?")
print ("peras" in estoqueFrutas1)
print()
print ("Tem mangas no estoque de Frutas 1?")
print ("mangas" not in estoqueFrutas1)
print()
```

Listagem 8

RETORNANDO ELEMENTOS ALEATÓRIOS

No exemplo anterior, nós estudamos uma situação onde havia uma lista de contatos que deveriam ser retornados. Numa situação real, o nosso usuário talvez não se importe a ordem para retornar as ligações. Assim, nós podemos utilizar um função semelhante ao pop(), porém, esta não recebe parâmetro e o valor retornado será qualquer um, em qualquer ordem.

Em um primeiro momento pode parecer não haver lógica essa função, mas na prática, é muito comum nós termos uma lista de tarefas e iremos resolver sem se importar com a ordem. Nessas situações, utilizar a função **popitem()** evita o trabalho de utilizar um looping e ao acessarmos um elemento, este será removido imediatamente.

```
#dicionario = {'1ra_chave':'1ro_valor', '2a_chave':'2do_valor'}
#Chave = Nome
#Valor = Telefone

retornar_ligacao = {
    "Andre": 30301122,
    "Carlos": 33547877,
    "Ana": 33381245,
    "Timoteo": 36458899
}

print("Ligações pendentes", retornar_ligacao)

retornar_ligacao.popitem() # Exclui o ultimo par (chave/valor) do dicionario ("Timoteo": 36458899)
print("Ligações pendentes", retornar_ligacao)

retornar_ligacao.popitem() # Exclui o ultimo par (chave/valor) do dicionario ("Ana": 33381245)
print("Ligações pendentes", retornar_ligacao)
```

Listagem 9

ACESSO A CHAVES INEXISTENTES

É comum nós tentarmos acessar um elemento de um dicionário que já tenha sido removido ou então, um elemento que achávamos que estaria contido. Nessas situações, será levantado uma exceção nos informando que estamos tentando fazer acesso a uma chave que não existe.

Atribuindo elementos com o mesmo valor a um outro dicionário

Nos momento em que precisarmos atribuir todos os elemento de um dicionário para outro, nós não podemos simplesmente atribuir o dicionário x e dizer que este receber y. Nessa situação, somente a referência seria copiada e não os elementos propriamente.

Logo, no momento em que fizéssemos acesso a um elemento e alterássemos o seu valor, esta alteração estaria visível em ambas as variáveis, até porque, ambas apontam para uma mesma estrutura.

Logo, se desejarmos duplicar os elementos e não somente atribuir a referência, temos a nossa disposição uma função específica.

MESCLANDO ITENS DE 2 DICIONÁRIOS (*intersecção de conjuntos*)

É comum na manipulação de dicionários a necessidade de unir um conjunto de elementos a um outro conjunto. Isso seria a intersecção de conjuntos, ou seja, os elementos de b são copiado para o elemento a. Para isso, nós temos um método de nome **update()**.

A seguir, temos um exemplo onde os elementos do conjunto "b" são copiados para o conjunto "a".

```
#dicionario = {'1ra_chave':'1ro_valor', '2a_chave':'2do_valor'}
#Chave = Frutas
#Valor = Quantidade

estoqueFrutas1 = {'kiwis': 430,
                  'bananas': 312,
```

```
'laranjas': 525,
'peras': 217
}

estoqueFrutas2 = {'mangas': 130,
'abacaxis': 150,
'uvas': 325,
'ameixas': 517
}

print("Calculo da Soma (total) de frutas no estoque de frutas1 + estoque de frutas2")
estoqueFrutas1.update(estoqueFrutas2)
print("Total de frutas no estoque 1 ",estoqueFrutas1)
```

Listagem 10

Vale observar que os elementos de um conjunto não são adicionados a outro conjunto, o que ocorre é somente uma cópia. Logo, as alterações feitas nos elementos do conjunto alvo não serão refletidas no conjunto receptor.

ITERANDO DICIONÁRIOS

Basicamente, um dicionário pode ser iterado seguindo a estrutura já estudada. O que o diferencia é que ao invés de um valor ser atribuído a uma única variável a cada ciclo, nós agora poderemos fazer uso de duas variáveis e assim, para a primeira será atribuído a chave e para a segunda variável será atribuído o valor.

Nos exemplos a seguir, estudaremos cada situação:

```
#dicionario = {'1ra_chave':'1ro_valor', '2a_chave':'2do_valor'}
#Chave = Frutas
#Valor = Quantidade

estoqueFrutas1 = {'kiwis': 430,
'bananas': 312,
'laranjas': 525,
'peras': 217
}

for frutas in estoqueFrutas1.keys():
    print("Listagem das frutas no estoque 1 = ", frutas)
print()
print(" Lista de frutas: ", list(estoqueFrutas1.keys()))
```

Listagem 11: Listagem das frutas(chaves) do dicionário Estoque de frutas 1

Neste exemplo nós imprimimos todas as chaves do nosso dicionário, porém, como imprimir os valores?

```
#dicionario = {'1ra_chave':'1ro_valor', '2a_chave':'2do_valor'}
#Chave = Frutas
#Valor = Quantidade

estoqueFrutas1 = {'kiwis': 430,
'bananas': 312,
'laranjas': 525,
'peras': 217
}
```

```
for quantidade in estoqueFrutas1.values():
    print("Listagem das quantidades de frutas no estoque 1 = ", quantidade)
print()
print(" Lista de quantidade: ", list(estoqueFrutas1.values()))
```

Listagem 12: Listagem das quantidades(valor) do dicionário Estoque de frutas 1

Quando nós desejarmos iterar e obter a cada ciclo a lista de chaves, nós podemos utilizar a variável que contém a referência para o dicionário. Agora, se nós desejarmos que o valor seja atribuído a cada ciclo para a variável, nós temos que iterar utilizando a função de nome `values()` que retorna um objeto do tipo `iterator`.

CONVERTENDO DICIONÁRIO EM LISTA

As vezes torna-se necessário a conversão entre estruturas diferente, como exemplo, converter um dicionário em uma lista. Internamente, o Python manipula um dicionário de maneira muito parecida a uma lista. De fato, um dicionário, como nós já estudamos, é uma lista, porém uma lista especializada.

Há **3 funções** para extrair uma lista de um dicionário.

Função	Descrição
keys()	✓ Retorna a lista de chaves contidas no nosso dicionário. ✓ Extraem no formato de lista os dados contidas em todo o dicionário, ou seja, uma lista contendo as chaves
values()	✓ Retorna a lista de valores. ✓ Extraem no formato de lista os dados contidas em todo o dicionário, ou seja, uma lista contendo os valores.
Items()	✓ Retorna uma lista de tuplas. ✓ Assim, o valor retornado será uma lista contendo tuplas, onde cada tupla será composta pela chave e pelo valor mapeado respectivamente. ✓ Essa é uma maneira bastante eficaz para que possamos rapidamente extrair todas as informações contidas na estrutura dicionário.

A seguir, temos uma tabela que contém as funções que extraem as informações de um dicionário.

Funções que retornam listas de dicionários

```
dict.keys()
```

Retorna a lista de chaves contidas em um dicionário.

```
dict.values()
```

Apelidos and cópias

Como os dicionários **são mutáveis**, você precisa estar ciente de apelidos (ou aliasing, como vimos com listas). Sempre que duas variáveis se referem ao mesmo objeto no dicionário, a mudança de uma afeta a outra.

Por exemplo, **opposites** é um dicionário que contém pares de opostos.

```
#dicionario = {'1ra_chave':'1ro_valor', '2a_chave':'2do_valor'}
#Chave =
#Valor =
antonimo= {'up': 'down', 'right': 'wrong', 'true': 'false'}
alias = antonimo

print("Conteudo do dicionario antonimo = ",antonimo)
print("Uso do alias ", alias is antonimo)

alias['right'] = 'left'
print("Correção do valor de right =" , antonimo['right'])
print()
print("Conteudo do dicionario antonimo = ",antonimo)
print()
```

Listagem 13

Como você pode notar do operador `is`, `alias` e **opposites** se referem ao mesmo objeto.

Se você deseja modificar um dicionário e manter uma cópia do original, use o método **copy** de dicionários. como **acopy** é uma cópia do dicionário, mudanças nesse dicionário não afetarão o original.

```
#dicionario = {'1ra_chave':'1ro_valor', '2a_chave':'2do_valor'}
#Chave =
#Valor =
antonimo= {'up': 'down', 'right': 'wrong', 'true': 'false'}

copiaAntonimo = antonimo.copy()
print("Conteudo do dicionario Copia Antonimo = ",copiaAntonimo)
```

Listagem 14

CONVERSÃO - LISTA PARA DICIONÁRIO

Existe várias formas para que listas sejam convertidas em dicionários, no entanto, não podemos esquecer que um dicionário é um elemento que contém chave-valor. Não podemos converter uma única lista em um dicionário sem chaves, até seria possível o oposto, porém, não há muita razão para este tipo de abordagem.

É mais comum convertermos duas listas que contém a mesma quantidade de elementos em um dicionário. Assim, se nós tivermos em uma lista um conjunto de elementos que representem as chaves e noutra um conjunto de elementos que representem os valores, nós podemos converter essas listas em dicionário.

A função built-in `zip` do Python retorna uma lista contendo tuplas, onde o primeiro valor é o da primeira lista, e o segundo valor da tupla, corresponde a segunda lista. Assim, com a função `zip` e a coerção de tipos, conseguimos obter um dicionário através de duas listas.

A seguir, nós criamos 2 listas e em seguida, convertemos as listas em uma lista de tuplas e depois, convertemos as listas em um dicionário.

```
>>> d1 = ['aaa','bbb','ccc']
>>> d2 = ['1','2','3']
>>> list(zip(d1, d2))
[('aaa', '1'), ('bbb', '2'), ('ccc', '3')]
>>> dict(zip(d1, d2))
{'aaa': '1', 'bbb': '2', 'ccc': '3'}
```

A função `zip` faz parte da programação funcional implementada pelo Python. Assim, essa é uma das ferramentas que foram implementadas na linguagem e que trabalham muito bem com o paradigma de orientação a objetos.

Estrutura de dados Árvore

Estrutura de dados disposta numa hierarquia. A estrutura de directórios do nosso computador é uma árvore. Execute no seu terminal o comando **`tree`** e terá algo parecido com:

```
nelzio@nelzio:~/devProjects/A$ tree
.
├── A
│   ├── AA
│   │   ├── AAA
│   │   ├── AAB
│   │   └── AAC
│   ├── AB
│   │   ├── ABA
│   │   ├── ABB
│   │   └── ABC
│   └── AC
│       ├── ACA
│       └── ACD
└── B
    ├── BA
    │   ├── BAA
    │   ├── BAB
    │   └── BAC
    ├── BB
    │   ├── BBA
    │   └── BBB
    └── BC
        ├── BCA
        ├── BCB
        ├── BCC
        └── BCD
```

Figura 3

O **ponto(.)** antes de A e B é a raiz da árvore e A e B filhos ou nós. Toda árvore tem uma única raiz. A estrutura de dados Árvore tem no **nível 0 a sua raiz**, e os **elementos de outros níveis são consideradas filhas**.

Pensemos na estrutura de dados árvore como uma árvore familiar. Onde todos os descendentes (filhos ou **nó**) de um avô (raiz) têm mesmo génese. Repare de novo na imagem acima.

Para perceber melhor veja esta imagem abaixo:

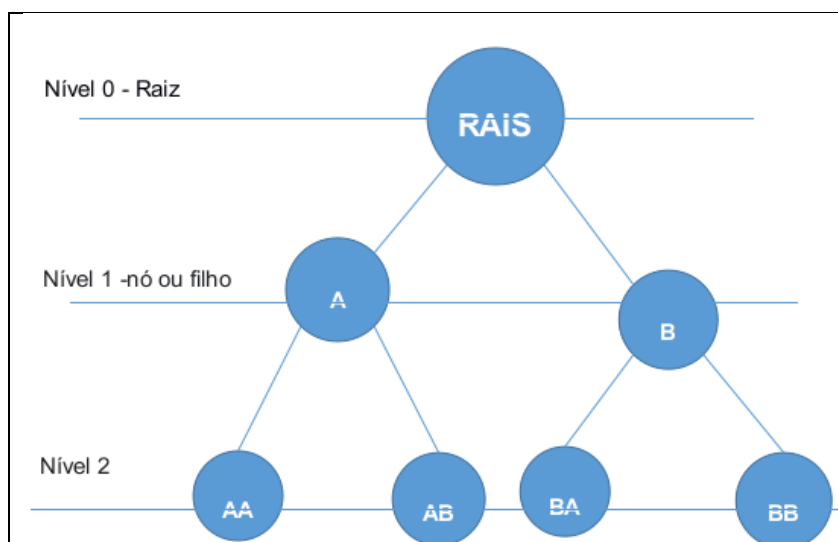


Figura 4

Glossário

chave

Um item de dados que é *mapeado* para um valor em um dicionário. As chaves são utilizadas para acessar os respectivos valores em um dicionário.

dicionário

Uma coleção de pares chave-valor que mapeiam chaves em valores. As chaves podem ser de qualquer tipo imutável e os valores podem ser de qualquer tipo.

gráfico de chamadas

Um gráfico consistindo de nós que representam chamadas de funções (ou invocações), e arestas (linhas com setas) mostrando que chamadas deram origem a outras chamadas.

memorando

O armazenamento temporário de valores pré-computadas para evitar a duplicação da mesma computação.

par chave-valor

Um dos pares de elementos de um dicionário. Os valores são acessados em um dicionário pelas suas chaves.

tipo de mapeamento

Um tipo de mapeamento é um tipo de dados composto por uma coleção de chaves e valores associados. O único tipo de mapeamento nativo do Python é o dicionário. Dicionários implementam o tipo de dados abstrato — *vetores associativos* <http://en.wikipedia.org/wiki/Associative_array>.

Bibliografia

Python, Escreva seus primeiros programas, Felipe Cruz, Casa do Código.

Curso Introdutório de Python, Grupy-Sanca, 09 de maio de 2019

Tutorial Python, Release 2.4.2. Guido van Rossum, Fred L. Drake, Jr., editor. Tradução: Python Brasil

Curso Python 3. Juracy Filho e Leonardo Leitão. Cod3r.com.br

<https://panda.ime.usp.br/pensepy/static/pensepy/11-Dicionarios/dicionarios.html>

<https://www.devmedia.com.br/python-trabalhando-com-variaveis/38644>

<https://cadernodelaboratorio.com.br/2017/12/01/variaveis-em-python3/>

Parte 2 – Prática (mão na massa)!!!

Para as atividades práticas vamos utilizar o VS Code, que deve estar previamente configurado para executar o python)

Para a criação dos arquivos (com extensão .py) vamos utilizar o editor de texto

Para executar, deve utilizar o interpretador do VS Code, temos 2 opções

- a) Apertar o Botão direito e selecionar a opção **RUN PYTHON TERMINAL**
- b) No menu superior, selecionar Debug -> **START WITHOUT DEBUGGING**

Para limpar o console, utilizar o comando **clear** ou **cls**

2) Criação do Projeto no VS CODE

- 1) Para isto crie uma pasta no seu pendrive com o nome **IPE-Python-2022**.
- 2) Dentro do **IPE-Python-2022**, criar a pasta **Aula08-Python**
- 3) Utilizando o VS CODE acesse a pasta criada (**IPE-Python-2022/ Aula08-Python**)
- 4) Para isso, acione a opção **File > New > File e crie os seguintes** (exemplo80.py, exemplo81.py, exemplo82.py, exemplo83.py e exemplo84.py), para melhor organização (**BOA PRÁTICA!!!**)

Exemplo 80

PASTA: IPE-Python-2022/ Aula08-Python
arquivo: exemplo80.py

```

1  lista1 = [1,2,3,4,10,8,9, 5, 0]
2  print("Elementos da lista 1")
3  print(lista1)
4  type(lista1)
5  print("Elemento da 1ra posição =", lista1[0])
6  print("Elemento da 5ra posição =", lista1[4])
7  print("Soma em uma lista ", (lista1[0] + lista1[4]))
8
9  lista_caracteres = ["a", "b", "c", "d", "e", "f", "g"]
10 print("Conteudo da lista de caracteres =", lista_caracteres)
11 print("Elemento da 1ra posição =", lista_caracteres[0])
12 print("Elemento da 2da posição =", lista_caracteres[1])
13 print("Elemento da 3ra posição =", lista_caracteres[3])
14 print("Elemento da 6ta posição =", lista_caracteres[5])
15
16 lista2= list("Python")
17 print ("Elemento da lista 2 =",lista2)
18
19 lista3= list(("Python",))
20 print ("Elemento da lista 3 =",lista3)
21

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

Conteudo do dicionario antonimo = {'up': 'down', 'right': 'left', 'true': 'false'}

Conteudo do dicionario Copia Antonimo = {'up': 'down', 'right': 'left', 'true': 'false'}

PS G:\IPE-Python2019>

Figura A: exemplo80.py

Exemplo 81

PASTA: IPE-Python-2022/ Aula08-Python
arquivo: exemplo81.py

```

3  #Valor = Numero em espanol
4
5  english2spanish = {}
6  english2spanish['one'] = 'uno'
7  english2spanish['two'] = 'dos'
8  english2spanish['three'] = 'tres'
9  english2spanish['four'] = 'cuatro'
10
11 print("Conteudo do dicionario Ingles para Espanhol = ", english2spanish)
12
13 english2spanish2 = {'three': 'tres',
14                    'one': 'uno',
15                    'two': 'dos',
16                    'four': 'cuatro'}
17
18
19 for item in english2spanish2:
20     print("Listagem das chaves do dicionario Ingles para Espanhol 2 = ", item)
21
22 print()
23 for item in english2spanish2.values():
24     print("Listagem dos valores do dicionario Ingles para Espanhol 2 = ", item)
25
26 print()
27 #print("Conteudo do dicionario Ingles para Espanhol 2 = ",english2spanish)
28
29 valor = english2spanish['two']
30 print("Conteudo da chave two = ", valor)
31

```

Figura B: exemplo81.py

Exemplo 82

PASTA: IPE-Python-2022/ Aula08-Python
arquivo: exemplo82.py

```

1  #dicionario = {'1ra_chave':'1ro_valor', '2a_chave':'2do_valor'}
2  #Chave = Nome
3  #Valor = Telefone
4
5  retornar_ligacao = {
6      "Andre": 30301122,
7      "Carlos": 33547877,
8      "Ana": 33381245,
9      "Timoteo": 36458899
10 }
11
12 print("Ligações pendentes", retornar_ligacao)
13
14 for nome in retornar_ligacao:
15     print("Listagem de nomes do dicionario retornar Ligação = ", nome)
16
17 print()
18 for telefone in retornar_ligacao.values():
19     print("Listagem dos numeros do dicionario retornar Ligação = ", telefone)
20
21 print()
22 print("Ligações pendentes", retornar_ligacao)
23
24 retornar_ligacao.popitem() # Exclui o ultimo par (chave/valor) do dicionario ("Timoteo": 36458899)
25 print("Ligações pendentes", retornar_ligacao)
26
27 retornar_ligacao.popitem() # Exclui o ultimo par (chave/valor) do dicionario ("Ana": 33381245)
28 print("Ligações pendentes", retornar_ligacao)
  
```

Figura C: exemplo82.py

Exemplo 83

PASTA: IPE-Python-2022/ Aula08-Python
arquivo: exemplo83.py

```

1  #dicionario = {'1ra_chave':'1ro_valor', '2a_chave':'2do_valor'}
2  #Chave = Estado
3  #Valor = Capital
4
5  dicEstadoCapital= {"Rio de Janeiro":"Rio de Janeiro",
6                     "São Paulo":"São Paulo",
7                     "Minas Gerais": "Belo Horizonte",
8                     "Bahia":"Salvador",
9                     "Amazonas":"Manaus"}
10
11
12 print(" A capital de Rio de Janeiro é ", dicEstadoCapital["Rio de Janeiro"])
13
14 for chave in dicEstadoCapital:
15     print("Listagem das chaves do dic Estado Capital = ", chave)
16
17 print()
18 for valor in dicEstadoCapital.values():
19     print("Listagem dos valores do dicionario Ingles para Espanhol 2 = ", valor)
20
21 print(" Lista de estados: ", list(dicEstadoCapital.keys()))
22 print()
23 print(" Lista de capitais: ", list(dicEstadoCapital.values()))
  
```

Figura D: exemplo83.py

Exemplo 84**PASTA: IPE-Python-2022/ Aula08-Python****arquivo: exemplo84.py**

```
#dicionario = {'1ra_chave':'1ro_valor', '2a_chave':'2do_valor'}
#Chave = Frutas
#Valor = Quantidade

estoqueFrutas1 = {'kiwis': 430,
                  'bananas': 312,
                  'laranjas': 525,
                  'peras': 217
                  }

estoqueFrutas2 = {'mangas': 130,
                  'abacaxis': 150,
                  'uvas': 325,
                  'ameixas': 517
                  }

print("Conteúdo completo do dicionário (estoque) = " , estoqueFrutas1)

print("Conteúdo completo do dicionário (estoque) = " , estoqueFrutas2)

del estoqueFrutas1['peras']

print("Conteúdo do dicionário (estoque), após a exclusão do chave/valor = pera " , estoqueFrutas1)

estoqueFrutas1['bananas'] = estoqueFrutas1['bananas'] + 200

numItens = len(estoqueFrutas1)
print("Numero de itens = " , numItens)
print("Estoque de bananas = " , estoqueFrutas1['bananas'])

print("Estoque total de frutas = " , estoqueFrutas1)

print ("Tem peras no estoque de Frutas 1?")
print ("peras" in estoqueFrutas1)
print()
print ("Tem mangas no estoque de Frutas 1?")
print ("mangas" not in estoqueFrutas1)
print()

for frutas in estoqueFrutas1.keys():
    print("Listagem das frutas no estoque 1 = " , frutas)
print()
print(" Lista de frutas: " , list(estoqueFrutas1.keys()))

print()
for quantidade in estoqueFrutas1.values():
    print("Listagem das quantidades de frutas no estoque 1 = " , quantidade)
print()
print(" Lista de quantidade: " , list(estoqueFrutas1.values()))
print()
for i in estoqueFrutas1:
    print("Listagem key e valor")
    print(i, estoqueFrutas1.items())
print()
print("Calculo da Soma (total) de frutas no estoque de frutas1 + estoque de frutas2")
estoqueFrutas1.update(estoqueFrutas2)
```

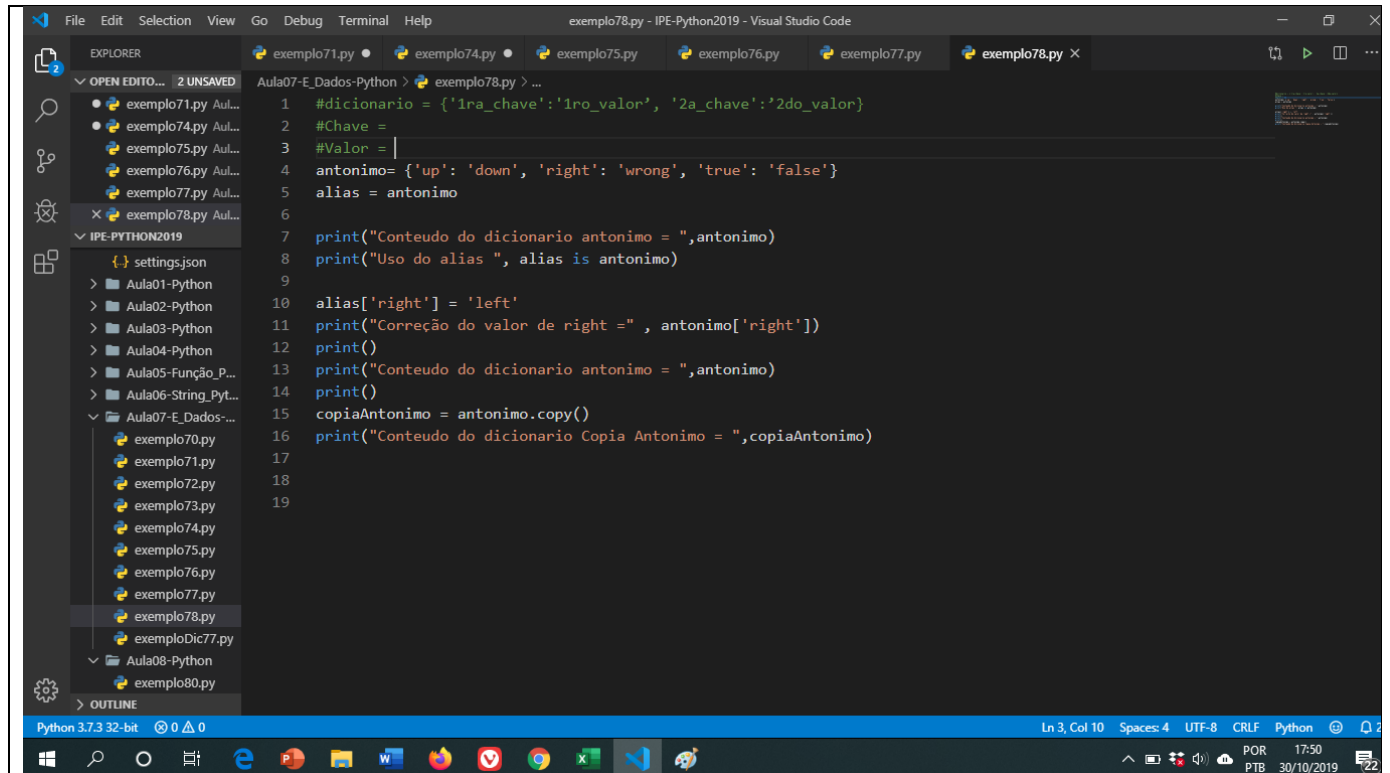
```
print("Total de frutas no estoque 1 ",estoqueFrutas1)
```

Figura E: exemplo84.py

Exemplo 85

PASTA: IPE-Python-2022/ Aula08-Python

arquivo: exemplo85.py



```
1 #dicionario = {'1ra_chave':'1ro_valor', '2a_chave':'2do_valor'}
2 #Chave =
3 #Valor =
4 antonimo= {'up': 'down', 'right': 'wrong', 'true': 'false'}
5 alias = antonimo
6
7 print("Conteudo do dicionario antonimo = ",antonimo)
8 print("Uso do alias ", alias is antonimo)
9
10 alias['right'] = 'left'
11 print("Correção do valor de right = ", antonimo['right'])
12 print()
13 print("Conteudo do dicionario antonimo = ",antonimo)
14 print()
15 copiaAntonimo = antonimo.copy()
16 print("Conteudo do dicionario Copia Antonimo = ",copiaAntonimo)
17
18
19
```

Figura F: exemplo85.py